Week 8 Sentiment Analytics with VADER

FINS5545: Financial Market Data Literacy

Today's Lecture

We use the VADER, Valence Aware Dictionary and sEntiment Reasoner to derive sentiment scores from text data

- We will walk through the VADER model, how it works, it's lexicon, etc.
- VADER has 5 'rules' which allow it to produce a sentiment based score from text
- As an example, we look at movie reviews, derive a sentiment score and compare it to human reviewers

A rules-based sentiment score is produced by a model using a pre-specified lexicon.

Setup Before We Start

Download `week08_student_folder.zip` from Ed and unzip it into your `fins2026` folder

```
fins2026/
  week7/
  week8/           <- appears, with everything already in it
    vader_model/
      01_what_is_a_sentiment_score.py ... 10_extend_and_test_vader.py
      vader_tools.py
      provided_data/   the reviews and the rating sheets, already here
      output/          already created, empty, ready
```

As usual, everything is set-up for you ...

Packages Used Today

All six are already listed in the repository `requirements.txt`, so if your environment is set up from the course repository you will have them. If you cannot run the code today, ask your AI agent to install the packages, or follow the below

```
pip install vaderSentiment pandas matplotlib pyarrow scikit-learn
    wordcloud
```

Package	Job today
`vaderSentiment`	the sentiment model itself
`pandas`, `pyarrow`	tables and the frozen review file
`matplotlib`, `wordcloud`	the figures
`scikit-learn`	counting agreements and disagreements

If Python reports a missing package, paste the full error to your AI coding agent and ask it to install only the package the error names.

Glossary (1) — Sentiment and Scores

Keep this open while we work, because every word here is used later today.

Term	Plain meaning
sentiment	the tone of a piece of writing, how positive or negative it is
sentiment analysis	scoring that tone with a stated model
sentiment score	the number the model returns; higher means more positive
positive	the score is above the cut-off, here $+0.05$
negative	the score is below the cut-off, here -0.05
neutral	the score lies between the two cut-offs, so no direction is claimed
lexicon	a fixed list of words; a *sentiment* lexicon also gives each word a tone
valence	the number a lexicon attaches to one word, from -4 to $+4$ in VADER
token	one unit of text after splitting; here, one word or emoticon
coverage gap	a word the writing uses that the lexicon has no entry for

Glossary (2) — The VADER Model

These are the parts of VADER itself, and the four numbers it returns.

Term	Plain meaning
VADER	Valence Aware Dictionary and sEntiment Reasoner
rule	an instruction that adjusts a word's valence using the words around it
booster	a word that raises the next word's valence, such as `very`
dampener	a word that lowers it, such as `marginally`
negation	a word that flips the sign of a valence, such as `not`
stored phrase	a multi-word entry scored whole, such as `the bomb`
normalisation	rescaling a total so it cannot leave the range -1 to 1
compound	the summed adjusted valences after normalisation, -1 to $+1$
pos, **neu**, **neg**	the shares of the text carrying positive, no, and negative valence
threshold	the cut-off that turns a score into a label, here ± 0.05
standard deviation	how far apart a set of numbers are, used here for one term's ten ratings

Sentiment Analytics

Three Families of Sentiment Model

Family	How it works	Main limitation
Word list	Count words from a positive list and a negative list.	Cannot tell a strong word from a mild one.
Dictionary plus rules	Score each known word, then adjust for the words around it.	Needs the word to be in the dictionary.
Trained model	Learn the scoring from labelled examples.	Needs labelled training data and is harder to inspect.

This course focusses on ‘Dictionary plus rules’.

Uses and Limits of a Sentiment Score

- A score compresses a long piece of writing into a number you can sort, average, and plot beside a price.
- That compression throws away *who* is being written about, *what* is being claimed, and whether the writer meant it (sarcasm?)
- A score is a measurement produced by a model, so it carries that model's blind spots into everything computed from it – modelling risk

A sentiment score measures word choice, not true meaning or implied meaning.

The VADER Model

The Two Parts of VADER

VADER stands for Valence Aware Dictionary and sEntiment Reasoner. It has two parts and no training steps (i.e., NOT a machine learning model) (Hutto and Gilbert, 2014).

Step	What happens
1. Look up	each known word gets a base **valence** from the dictionary
2. Adjust	five rules change that valence using the "words around it"
3. Add	the adjusted valences are summed into a total
4. Rescale	the total is re-scaled into the range -1 to 1
5. Report	four are produced, `neg`, `neu`, `pos`, `compound`

Every one of those steps is simple arithmetic you can redo by hand – we will go over some examples in class. Remember end goal is going from raw text to these 4 numbers ...

What the Four Numbers Mean

Every time you sentiment score a piece of text, VADER returns these four numbers – nothing else – this is the Stage 3 model output.

Field	Range	What it is
`pos`	0 to 1	the share of the text made up of positive-scoring words
`neu`	0 to 1	the share made up of words the dictionary has no entry for
`neg`	0 to 1	the share made up of negative-scoring words
`compound`	-1 to $+1$	an overall score for the text, negative for negative writing and positive for positive writing

For `The earnings report was good.` VADER returns `pos` $= 0.420$, `neu` $= 0.580$, `neg` $= 0.000$, `compound` $=$ **0.4404**. Each of the three is rounded to three decimals, so across the 2,000 film reviews they add to 1.000 for 74% of them and to 0.999 or 1.001 for the rest.

`pos`, `neu`, and `neg` are three shares of the same text, so they add to 1 apart from rounding. `compound` is separate and is the one to report.

Where the Dictionary Numbers Came From

The authors did not write the numbers themselves. They ran a rating exercise (Hutto and Gilbert, 2014).

- They drew up a shortlist of more than 9,000 words and symbols that might belong in a sentiment dictionary, taken from older word lists and from the emoticons, abbreviations, and slang people use online.
- Nothing was in the dictionary yet. Every one of those 9,000 was put to ten people, who rated it from -4, extremely negative, to $+4$, extremely positive.
- Each term therefore has ten ratings. A term was kept only if the mean of those ten was not zero, and only if they had a standard deviation below 2.5, which is a measure of how far apart the ten raters were. Terms the raters disagreed about were dropped.
- Just over 7,500 of the original 9,000 survived, and those are the entries in the dictionary today.

Each valence is the mean of ten human ratings, so the dictionary records what people agreed a word means.

VADER's Results in the Paper

The authors scored 4,200 short tweets. Each was already rated by at least 20 trained people, and the average of their ratings is the **ground truth** VADER is graded against.

- **Correlation** measures whether VADER's compound score rises and falls with the human average, where 1 means they move together exactly.
- **F1** turns each score into a positive, negative, or neutral label, then measures how often that label is right, from 0 to 1. It combines precision and recall, and 1 is a perfect score.

On 4,200 tweets	Correlation with ground truth	F1 score
VADER	0.881	**0.96**
One human rater	0.888	0.84

On correlation VADER (0.881) all but ties one person (0.888). Its F1 (0.96) beats one person (0.84) because that label is graded against a 20-person average, and one rater disagrees with the average more than VADER's rules do. VADER matches the crowd, it is not smarter than a human.

The F1 of 0.96 is for short social-media text scored against a 20-person average. Do not carry it across to financial filings, news, or long reviews.

Inside the VADER Dictionary

Script 02 opens the dictionary the installed package is using

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
analyzer.lexicon["good"]          # 1.9
len(analyzer.lexicon)             # 7506
```

Property of the dictionary	Value
terms	**7,506**
valence range	−3.9 to 3.4
terms with a negative valence	4,169
terms with a positive valence	3,337

A row in this dictionary is a term and a number (between −4 and 4.

Valence Records Strength

A positive-or-negative word list would treat every word in each column the same (i.e., give each negative or positive word a similar score) A valence-aware dictionary allows for hetergeneity in the 'strength' of the negative or positive sentiment of the word

Term	Valence	Term	Valence
okay	**0.9**	bad	**-2.5**
good	**1.9**	terrible	**-2.1**
great	**3.1**	awful	**-2.0**
excellent	**2.7**	horrible	**-2.5**

This is the difference the VADER authors were after. Writing that the food was `okay` and writing that it was `great` both count as a positive word, but they do not carry the same strength.

Emoticons, Slang, and Abbreviations

VADER was built for social-media writing, so the dictionary holds terms the older word lists left out.

Term	Valence	Term	Valence
`:)`	2.0	`:(`	-1.9
`lol`	1.8	`meh`	-0.3
`rofl`	2.7	`smh`	-1.3

This is also a warning about fit. A model whose dictionary contains `rofl` was designed for a kind of writing that looks nothing like an annual report.

Coverage Gaps in Finance Vocabulary

Look up finance vocabulary in ... many potentially crucial words are missing!!

Term	In the dictionary	Contribution to the score
hawkish	no	0
dovish	no	0
guidance	no	0
impairment	no	0
downgrade	no	0
debt	yes	**-1.5**

Five of these six contribute nothing, which is a **coverage gap** rather than a finding that ten people read `hawkish` and judged it neutral. The dictionary simply has no entry for it.

A zero can mean the model had no opinion or that it had no entry. Check which one before reporting it.

The Five Rules that VADER Uses

The Rules and Their Constants

A dictionary alone cannot tell `good` from `not good`. The rules read the words around a scored word and change its valence. Each constant below was measured by the authors in a rating experiment, then written into the code.

Rule	Operation	Amount
Exclamation mark	add to the running total	+0.292 each, at most four
ALL-CAPS word	add to that word's valence	+0.733
Booster word	add to the next word's valence	+0.293
Dampener word	subtract from the next word's valence	−0.293
Negation	multiply the valence	$\times -0.74$
The word `but`	reweight the two halves	$\times 0.5$ before, $\times 1.5$ after

The base valence of `good` is 1.9. Every number in this section starts from there.

Rule 1, Exclamation Marks

An exclamation mark does not change the direction of a score. It makes whatever direction is already there stronger, by adding to the running total.

Sentence	Arithmetic	Total	Compound
The film was good.	1.9	1.900	**0.4404**
The film was good!	$1.9 + 0.292$	2.192	**0.4926**
The film was good!!!	$1.9 + 3 \times 0.292$	2.776	**0.5826**

Marks past the fourth are ignored, so a row of twenty of them counts as four. On a negative total the same amount is subtracted instead, which pushes the score further down rather than up.

Rule 2, Capitalisation

A capitalised word adds 0.733 to its own valence, but only when the rest of the sentence is not also capitalised.

Sentence	Arithmetic	Compound
The film was good.	1.9	**0.4404**
The film was GOOD.	$1.9 + 0.733$	**0.5622**
THE FILM WAS GOOD.	1.9, rule switched off	**0.4404**

When every word shouts, no word stands out, so the rule does nothing. This is the first reason the text has to reach the model with its capitals intact. Lower-casing a column switches this rule off for every row in it.

The capitalisation rule fires on the contrast between a shouted word and its neighbours, so it disappears the moment you lower-case the text.

Rule 3, Boosters and Dampeners

VADER keeps a list of 84 words that modify the word after them. Sixty raise the intensity and twenty-four lower it.

Boosters, 60 in the list	absolutely, amazingly, completely, deeply, enormously, especially, extremely, incredibly, particularly, purely, really, remarkably, so, substantially, totally, tremendously, unusually, utterly, very
Dampeners, all 24 of them	almost, barely, hardly, just enough, kind of, kinda, less, little, marginal, marginally, occasional, occasionally, partly, scarce, scarcely, slight, slightly, somewhat, sort of, sorta

Sentence	Arithmetic	Compound
The film was very good.	$1.9 + 0.293$	**0.4927**
The film was marginally good.	$1.9 - 0.293$	**0.3832**

Rule 4, Negation

A negation multiplies the valence by -0.74. The sign flips, and because 0.74 is below one, the result is weaker than the word was to begin with.

Sentence	Arithmetic	Compound
The film was good.	1.9	**0.4404**
The film was not good.	1.9×-0.74	**-0.3412**

The list holds 59 entries, with each shortened form appearing twice, once with the apostrophe and once without, so `wasn't` and `wasnt` both count.

ain't, aren't, can't, cannot, couldn't, didn't, doesn't, don't, hadn't, hasn't, haven't, isn't, mustn't, needn't, neither, never, none, nope, nor, not, nothing, nowhere, oughtn't, shan't, shouldn't, wasn't, weren't, without, won't, wouldn't

Any word ending in `n't` counts as a negation even if it is not on the list, which is one more reason the punctuation has to reach the model intact.

Rule 5, the Word But

Everything before `but` is halved and everything after it is multiplied by one and a half, because the second half of such a sentence usually carries the writer's actual view.

The film was good, but the ending was terrible.	
`good` sits before `but`, so 1.9×0.5	0.95
`terrible` sits after `but`, so -2.1×1.5	**-3.15**
total	**-2.20**
compound	**-0.4939**

The sentence opens with praise and still scores negative. Without this rule the two words would have cancelled to roughly zero.

Stored Phrases

VADER keeps a short list of nine multi-word phrases, each with its own score. When one appears, VADER scores the whole phrase instead of the words inside it. `the bomb` is on that list, slang for excellent, and is stored with a positive valence of $+3$. The word `bomb` on its own carries the dictionary's negative valence of -2.2.

Sentence	Compound
This movie was the bomb.	**0.6124**
A bomb went off.	**-0.4939**

So the word `bomb` scores positive in the first sentence and negative in the second. VADER matched the stored phrase `the bomb` in the first and used its $+3$. In the second there is no stored phrase, so the word's own -2.2 applies. Nine phrases is a tiny list, so treat this as proof that word order can matter, not that VADER understands idiom in general.

Rule Words and Scored Words

The rules use their own word lists, kept separately from the valence dictionary. Look the rule words up in the dictionary and nothing comes back.

Word	In the valence dictionary	Which list holds it instead
very	no	the booster list
marginally	no	the dampener list
not	no	the negation list
never	no	the negation list

These words contribute no valence of their own. They are instructions for changing another words valence.

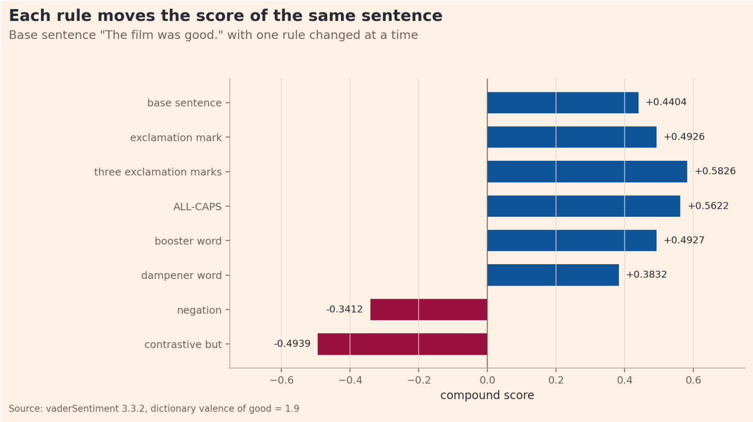
Every Rule on the Same Sentence

Each row takes `The film was good.` and applies one rule. The base score is **0.4404**.

Sentence	Rule	Compound
The film was good.	base	**0.4404**
The film was good!	exclamation	**0.4926**
The film was GOOD.	ALL-CAPS	**0.5622**
The film was very good.	booster	**0.4927**
The film was marginally good.	dampener	**0.3832**
The film was not good.	negation	**-0.3412**
The film was good, but the ending was terrible.	but	**-0.4939**

The first three changes raise the score, the dampener lowers it, and negation or `but` flip it below zero.

The Same Rules as a Bar Chart



Blue bars stay positive and maroon bars go negative. Only negation and but cross zero, so the other changes move the score without flipping its sign.

Combining the Valences

The Compound Score Formula

Let v_i be the base valence of word i from the dictionary and a_i its valence after the rules. Sum them into a total x.

$$x = \sum_i a_i \qquad \text{compound} = \frac{x}{\sqrt{x^2 + \alpha}}, \quad \alpha = 15$$

- A total of zero returns zero, so text with no scored words scores zero.
- The sign of x is kept, so adding does not change the direction.
- Large totals bunch up close to 1 and -1, so the output cannot leave that range however long the writing is.

The Role of Alpha

α is the constant in the compound formula compound $= x/\sqrt{x^2+\alpha}$, where the total x is the sum of the adjusted valences over the whole text from the previous slide. A larger x means the text carries more valence, and α sets how quickly the compound score climbs toward ± 1 as x grows. Each cell below is the compound score for that α (row) and that total x (column).

α	$x = 1$	$x = 3$	$x = 6$
1	0.7071	0.9487	0.9864
5	0.4082	0.8018	0.9370
15	**0.2500**	**0.6124**	**0.8402**
50	0.1400	0.3906	0.6470
200	0.0705	0.2075	0.3906

Saturation means the score has climbed so close to the ± 1 ceiling that a larger x barely moves it. With $\alpha = 1$ a total of only $x = 3$ already returns a compound score of 0.95, so a text with a little valence and one with a lot both read as near-extreme. A large α does the opposite. With $\alpha = 200$ a total of $x = 6$ returns a compound score of only 0.39, so scores rise in near-proportion to x and rarely reach ± 1. VADER's authors chose 15 as a balance, a tuning constant rather than a theoretical result, so record that choice beside the score.

From Words to a Compound Score

`The acting was great, but the plot was terrible!`

The word `but` halves the valence of every word before it ($\times 0.5$) and multiplies every word after it by 1.5, because the part after `but` usually carries the writer's real point.

Step	Arithmetic	Result
look up `great` in the dictionary	3.1	3.100
look up `terrible` in the dictionary	-2.1	−2.100
`great` is before `but`, so halve it	3.1×0.5	1.550
`terrible` is after `but`, so raise it	-2.1×1.5	−3.150
add the two adjusted valences	$1.55 + (-3.15)$	−1.600
total is negative, so the exclamation subtracts	$-1.6 - 0.292$	−1.892
rescale into the range −1 to 1	$-1.892/\sqrt{(-1.892)^2 + 15}$	−0.4389

Script 04 prints this table beside the package output, matching to four decimal places.

Where pos, neu, and neg Come From

Score `The acting was great, but the plot was terrible!` one word at a time. `compound` is never used here. As before, `but` halves the valence of a word before it ($\times 0.5$) and raises a word after it ($\times 1.5$), so `great` (3.1) becomes 1.55 and `terrible` (-2.1) becomes -3.15.

Word	Valence after rules	Pile	Adds to its pile
The	0	neutral	1
acting	0	neutral	1
was	0	neutral	1
great	$3.1 \times 0.5 = 1.55$	positive	$1.55 + 1 = \mathbf{2.550}$
but	0	neutral	1
the	0	neutral	1
plot	0	neutral	1
was	0	neutral	1
terrible	$-2.1 \times 1.5 = -3.15$	negative	$3.15 + 1 = 4.150$

The exclamation adds 0.292 to the larger pile, the negative one, giving $4.150 + 0.292 = \mathbf{4.442}$. The seven zero-valence words make the neutral pile 7. The denominator is the three piles added up, $2.550 + 4.442 + 7 = \mathbf{13.992}$.

$$\texttt{pos} = \frac{2.550}{13.992} = 0.182, \qquad \texttt{neg} = \frac{4.442}{13.992} = 0.317, \qquad \texttt{neu} = \frac{7}{13.992} = 0.500$$

In-Class Example
2,000 Film Reviews

The Film Review Data

We now apply the VADER model to film reviews ...

Property	Value
reviews	2,000, half positive and half negative
source	the collection released with Maas et al. (2011)
`review_text`	what the reviewer wrote
`true_label`	`pos` or `neg`, from the reviewer's own stars
`star_rating`	1 to 4 for negative, 7 to 10 for positive
median length	172 words

The label comes from the person who wrote the review, we can check if VADER's sentiment assignment matches

Text Signals Present in the Reviews

Before scoring, confirm the writing still contains what the rules read. If it did not, three of the five rules would be switched off before we started.

Signal in the text	Rule it switches on	Reviews	Share
ALL-CAPS word	capitalisation	1,012	51%
exclamation mark	punctuation	688	34%
a shortened `not`, as in `wasn't`	negation	1,165	58%
the word `but`	contrastive	1,430	72%

Only the HTML line-break tags the reviews were scraped with have been removed

Why the Text Is Not Cleaned

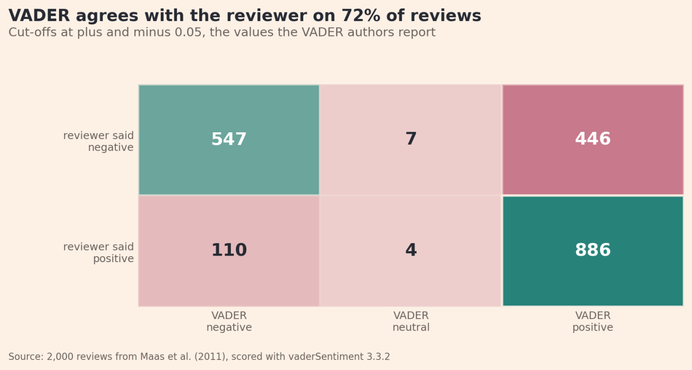
The column goes to the model exactly as the reviewer wrote it.

```
scores = reviews["review_text"].map(analyzer.polarity_scores).apply(pd.
    Series)
```

What was done to the text first	Scores that changed	Mean size of change
nothing	0	0.0000
lower-cased	226	0.0051
lower-cased and punctuation removed	**1,475**	**0.0995**

Lower-casing and stripping punctuation changes the score of 1,475 of the 2,000 reviews. Removing stop words would be worse again, because it deletes the negation and booster words the rules run on.

Agreement With the Reviewers



Reading across the top row, VADER called 446 of the 1,000 negative reviews positive. Reading the bottom row, it called only 110 of the positive reviews negative.

Error Rates by Direction

Each column is one group of reviews, split by the reviewer's own label. **Recall** is the share of a group that VADER labelled correctly — negative reviews it called negative, positive reviews it called positive.

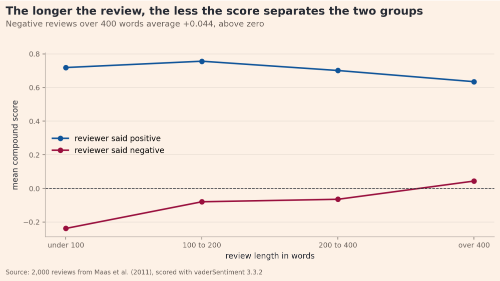
	Reviewer said negative	Reviewer said positive
mean compound	−0.0774	**0.7213**
median compound	−0.3428	**0.9628**
share VADER called positive	**45.1%**	88.7%
recall	**0.549**	0.887

The model finds positive writing reliably and misses negative writing about half the time. An average of these scores across a mixed collection will therefore read more positive than the collection actually is.

VADER recovers 88.7% of the positive reviews but only 54.9% of the negative ones, so any average built from these scores is biased upward.

Score Against Review Length

Reviews are sorted into four length bands. Each line is the average compound score of one group inside each band. A wide gap means the score separates the groups.



The blue line barely moves, while the maroon line climbs from -0.238 to $+0.044$ and crosses zero, so the average long negative review is scored positive and the gap between the two closes as reviews get longer.

Why Length Changes the Score

`compound` adds up the valence of every scored word before rescaling, so a longer document contributes more terms to the total. A long negative review still describes the plot, names the actors, and says what the film was trying to do, and those words carry positive valence, so the complaint is a smaller share of the text.

Review length	Mean score, negative	Mean score, positive	Agreement
under 100 words	−0.238	0.719	**78.9%**
100 to 200	−0.079	0.757	73.0%
200 to 400	−0.065	0.701	69.5%
over 400	+0.044	0.635	**65.7%**

The model is right about four short reviews in five and only two long reviews in three. Nothing about the writing got harder; the documents just got longer.

Document length changes the score on its own, so compound scores from documents of different lengths cannot be compared directly. A headline and an annual report are not on the same scale.

The Scored Words Behind Each Direction

The words that carried the score

Only words the VADER dictionary scores are shown, sized by how often they appear

Source: 2,000 reviews from Maas et al. (2011); vaderSentiment 3.3.2 dictionary

Only words the dictionary scores appear here, sized by how often they occur. Every other word was dropped from the figure, because a word with no valence played no part in the score.

Subject-Matter Words in the Dictionary

Look again at the two figures on the previous slide. Several of the largest words describe what a film is about rather than what the reviewer thought of it.

Word	What it usually describes in a review	Valence
horror	the genre	-2.7
kill	the plot	-3.7
murder	the plot	-3.7
war	the setting	-2.9
comedy	the genre	1.5
funny	the genre or the opinion	1.9

A favourable review of a war film is pushed negative by its own subject matter. The model cannot separate a description of the story from a judgement about it.

Application to Financial Text

Accounting Words in a General Dictionary

What the film reviews showed is not specific to films. A general-purpose dictionary scores subject-matter words as though they were opinions, and financial writing is full of subject-matter words that sound negative.

Word in a filing	What it usually means there	General dictionary treats it as
liability	an accounting category	negative
debt	a funding choice	negative
cost	a line item	negative
tax	a line item	negative

Loughran and McDonald (2011) found that almost three-quarters of the words a general dictionary flags as negative are not negative in financial writing.

Before using any sentiment dictionary on financial text, check which of its scored words are the subject matter of your documents.

Extending the Dictionary

Why the Gap Needs New Entries

Five of the six finance terms we looked up earlier — `hawkish`, `dovish`, `guidance`, `impairment`, and `downgrade` — were absent, and an absent term contributes nothing. Only `debt` had an entry. No amount of rule work fixes that, because the rules only adjust a valence that is already there.

- Negation can flip a known valence, but it cannot create one for `hawkish`.
- A booster can raise a known valence, but it cannot raise zero.
- So closing the gap means adding entries, which means deciding what numbers to add.

The VADER authors faced the same problem in 2014 and solved it by paying people to rate words. We repeat their method, changing only who does the rating.

Adding a dictionary entry is a modelling decision, so it needs a stated procedure and a written record of who decided what.

The Paper's Method With Model Raters

VADER in 2014	What we do here
a shortlist of over 9,000 words	a shortlist of 39 finance words
10 independent human raters	10 independent model raters
$9{,}000 \times 10 =$ over 90,000 ratings	$39 \times 10 = 390$ ratings
scale of -4 to $+4$	scale of -4 to $+4$
keep if the mean of its 10 ratings is not zero	keep if the mean of its 10 ratings is not zero
keep if those 10 ratings have a standard deviation below 2.5	keep if those 10 ratings have a standard deviation below 2.5

The ratings were collected once and frozen into `provided_data/raw_ratings/`, so the lecture needs no network call and no API key. Scripts 09 and 10 read them.

The Prompt Sent to Each Rater

```
You are one of several independent raters building a sentiment
dictionary for financial text. You will not see the other
raters' answers.

For each term below, give the sentiment valence it carries for
a reader of financial news and company filings.
  -4 extremely negative ... 0 neutral ... +4 extremely positive

- Rate the term on its own, with no sentence around it.
- Give the rating you think MOST OTHER RATERS would choose.
- 0 is a real answer.
- Judge the term as used in financial writing.
```

Three parts of that wording come from the paper. The scale, rating the term on its own, and asking what most other raters would say. The authors report that the last one tightened the spread between raters without moving the average.

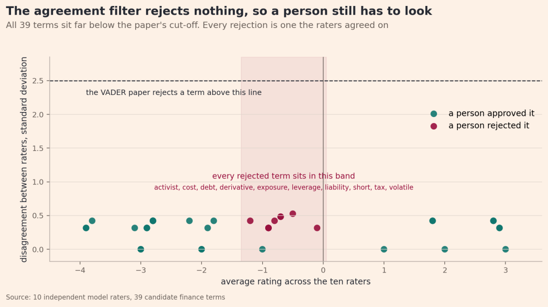
Ratings From the Ten Raters

Term	Mean of 10 ratings	Spread of 10 ratings	Lowest rating	Highest rating
going concern	**-3.9**	0.32	−4	−3
insolvency	**-3.9**	0.32	−4	−3
guidance cut	**-2.9**	0.32	−3	−2
headwind	**-1.8**	0.42	−2	−1
hawkish	**-1.0**	0.00	−1	−1
dovish	**1.0**	0.00	1	1
earnings beat	**2.9**	0.32	2	3
record revenue	**3.0**	0.00	3	3

The means look sensible and the spreads are narrow. Eight of the 39 terms drew the same rating from all ten raters. That agreement is the thing to examine next, rather than the thing to be reassured by.

How Much the Ten Raters Disagreed

One dot is one of the 39 terms. Left to right is the average of its ten ratings, so a dot on the far left is a term all ten raters called very negative. Up and down is how far apart those ten ratings were. A dot sitting on the bottom line means all ten raters gave that term the identical number.



The dashed line near the top is the paper's cut-off. A term whose raters disagreed that much would be thrown away. Nothing here comes close to it.

Why the Filter Rejects Nothing

Measure	Value
The paper throws a term away once its ten raters disagree by	2.5
Worst disagreement across all 39 of our terms	**0.527**
Terms where all ten raters gave the identical number	8
Terms our filter therefore rejected	**0**

The paper's test does real work when the raters are ten different people, because ten people genuinely disagree about what a word means. Our ten raters are ten runs of the same model. They were trained on the same text, so they answer almost identically, and a test built to catch disagreement finds none.

A Known Bias in the Ratings

Every term below passed the standard-deviation test. Every one is an accounting category or a position descriptor rather than a judgement.

Term	Mean of 10 ratings	Spread of 10 ratings	Why it was rejected
volatile	**-1.2**	0.42	a risk descriptor, not a judgement
cost	**-0.9**	0.32	an accounting category
debt	**-0.9**	0.32	an accounting category
short	**-0.9**	0.32	a position direction
liability	**-0.8**	0.42	an accounting category
tax	**-0.5**	0.53	an accounting category
derivative	**-0.1**	0.32	an instrument class

The Same Error, Found in 2011

Loughran and McDonald (2011) took a general-English sentiment dictionary, the kind built from everyday writing, and ran it over company annual reports. They then read the words it had flagged as negative. Almost three-quarters of the words that dictionary called negative are not negative in a company report.

They are the ordinary vocabulary of accounting.

`tax, cost, liability, capital, expense, depreciation, board, crude`

Every filing is full of these words whether the firm had a good year or a bad one, so a general dictionary reads almost any annual report as negative. That is the same mistake our ten raters just made, on the same kind of words, even though they were told the context was financial. You can overcome this with a careful prompt.

The ten raters agreed with each other and were wrong together, in a way documented 15 years ago. Agreement carried the bias through rather than catching it.

Scores Before and After the Extension

Sentence	Before	After	Change
The central bank sounded hawkish.	0.0000	**-0.2500**	−0.2500
Analysts issued a downgrade.	0.0000	**-0.5859**	−0.5859
The company reported an earnings beat.	0.0000	**0.5994**	+0.5994
The firm announced a buyback.	0.0000	**0.4215**	+0.4215
Management issued a guidance cut.	−0.2732	**-0.5994**	−0.3262
The auditor raised a going concern warning.	−0.3400	**-0.8957**	−0.5557
The firm reduced its debt.	−0.3612	−0.3612	0.0000
The stock is volatile.	0.0000	0.0000	0.0000

The last two rows use rejected terms and do not move, which is the check that the approval step controls what reaches the model.

Capabilities and Limits of VADER

It can	It cannot
score thousands of documents a second with no training data	tell a description of events from an opinion about them
give you every number and rule to inspect	score a word that is not in its dictionary
tell a strong word from a mild one	notice sarcasm, irony, or a comparison to another film
reproduce exactly, given the same text and version	tell you who or what the writing is about

References I

Clayton J. Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media*, 8(1):216–225, 2014. doi: 10.1609/icwsm.v8i1.14550.

Tim Loughran and Bill McDonald. When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks. *Journal of Finance*, 66(1):35–65, 2011.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA, 2011. Association for Computational Linguistics. URL https://aclanthology.org/P11-1015.